InterX 합류를 위한

과제 발표

"단순 구현보다 0.1%의 효율을 중시하는 AI Engineer"

주재홍 · 2026.04.24

kinduki123@gmail.com · github.com/flyingjoojak

저는 이런 엔지니어입니다

10년+ 개발 경험과 컴퓨터공학 전공의 결합

"돌아가는 AI"가 아니라 "효율로 증명되는 AI"를 만듭니다.

리소스 최적화

ONNX · 양자화 · 캐싱으로
하드웨어와 API 비용을 함께 절감

RapidOCR 70%↓ · SDXL 90%↓

고신뢰 RAG

Hybrid Search + Reranking 으로
토큰을 절약하며 정확도를 상승

관련성 +45% · 토큰 −30%

엔드-투-엔드 책임감

기획·모델·서빙·프론트·운영까지
직접 만들어 본 5개 프로젝트

비전 · LLM · 풀스택 · DevOps

커리어 하이라이트

엔드-투-엔드로 만든 5개 프로젝트 — 정량 지표로 증명

01
AI 광고 생성 파이프라인

소상공인 마케팅 자동화
로컬 추론으로 비용 0원

70% ↓ / 0원

210s → 30s · API 비용 0원

SDXL · Diffusers · FastAPI · FP16/INT8

02
RFP RAG 분석 시스템

입찰 제안서 자동 분석
근거 인용으로 환각 억제

+45% / −30%

관련성 +45% · 토큰 −30%

LangChain · ChromaDB · FlashRank · Ragas

03
약제 식별 시스템

조제 오류 방지를 위한
비전 검수 자동화

99.1%

750회 반복 실험

YOLOv11 · Optuna · Ensemble · OpenCV

04
디스코드 커뮤니티 플랫폼

활성도 기반 랭킹
가상 재화 경제 시스템

₩490K

100+ 서버 · DAU 300

Next.js · FastAPI · Pycord · PG · Redis

05
실시간 번역 오버레이

외국어 콘텐츠 실시간 번역
저사양에서도 상시 구동

70% ↓ / 0원

OCR 70% ↓ · 캐시로 비용 0원

PyQt6 · RapidOCR · ONNX Runtime

왼쪽 4개 = 효율(얼마나 줄였나) · 오른쪽 1개 = 실전성(얼마나 빨리 통했나)

지원 동기 — 왜 인터엑스인가

AI를 써서 무엇을 바꾸는가에 답이 있는 회사

01 글로벌 제조 AX 시장을 선도하는 기술 표준화

인터엑스가 지향하는 글로벌 표준 기반의 제조 혁신 비전에 깊이 공감합니다. 개인 사업을 운영하며 서비스의 시장 확장성과 운영 효율을 고민했던 경험을 바탕으로, 인터엑스의 솔루션이 전 세계 제조 현장에 유연하게 도입될 수 있도록 확장성 높은 백엔드 아키텍처를 구축하고 싶습니다.

02 제조 특화 생성형 AI 역량 강화

인터엑스의 '**제조 특화 생성형 AI**'는 복잡한 분석을 직관적 소통으로 바꾼 혁신 사례입니다. 그동안 쌓아온 LLM 및 RAG 관련 기술 역량을 투입하여, 현장 작업자가 전문가의 인사이트를 즉각적으로 얻을 수 있는 고도화된 인터페이스 개발에 기여하겠습니다.

03 자율 제조(EAX) 실현을 위한 데이터 최적화

Python, SQL 기반의 데이터 관리 능력과 FastAPI, Streamlit을 활용한 효율적인 백엔드 구축 경험을 보유하고 있습니다. 이를 통해 인터엑스가 지향하는 지능형 제조 생태계의 핵심인 데이터 처리 효율성을 극대화하겠습니다.

12가치 매칭 + 입사 후 포부

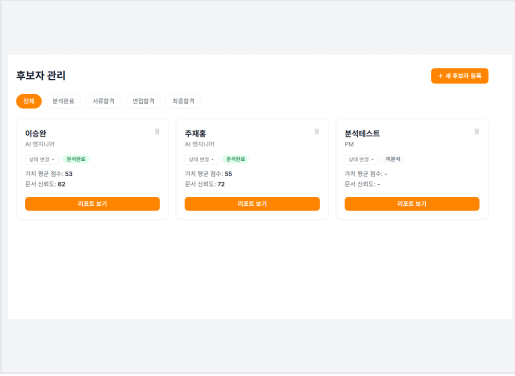
12가치 중 4개에 먼저 기여하겠습니다

끈기	약제 식별 프로젝트에서 750회 이상의 반복 실험과 정밀 튜닝을 거쳐 인식 정확도 99.1% 기록.
지속적개선	RFP RAG에 FlashRank 리랭킹 도입으로 노이즈 제거 + 연관성 높은 컨텍스트 선별. MobileNetv2 기반 RapidOCR을 ONNX로 최적화해 부하 최소화 및 추론 구현. 양자화·어텐션 최적화로 하드웨어 추론 성능 확보.
문제해결	조제실 내 오투약 리스크를 줄이기 위한 약제 식별 시스템 구축. 외국어 게임/영상 시청 시 발생하는 정보 격차를 실시간 번역 오버레이로 해소. 클릭 간격 표준편차 기반 행동 패턴 알고리즘으로 매크로 방지 로직 적용.
비판적사고	단순 LLM 호출에서 벗어나 Hybrid Search + FlashRank 리랭킹으로 토큰 낭비 절감과 신뢰도 확보. 운영비 절감을 위해 유료 API 대신 로컬 허깅페이스(SDXL) 모델 및 서빙 환경 구축. YOLOv11과 Faster R-CNN을 비교하고 Optuna로 하이퍼파라미터 최적화 수행.

포부 — 회귀 테스트 · 비용 계측 · 자가 검증 루프로 네 가치를 코드 레이어에 박아 넣겠습니다.

과제 오버뷰 — InterX MVP

FastAPI · LangGraph · Next.js 풀스택



데이터 흐름 (9단계)

① 이력서/포트폴리오 업로드 (PDF·JPG·PNG)

② Claude Vision OCR — 읽기 순서 강제

③ 구조화 추출 (6필드 JSON)

④ 12가치 매핑 (score·evidence·examples)

⑤ 자가검증 루프 (substring · token-overlap)

⑥ 문서 신뢰도 산출

⑦ 모순 탐지 (4축 · severity)

⑧ 사전 압박 질문 생성

⑨ 실시간 면접 (꼬리질문) → PDF 리포트

asyncio.Queue 단일 워커로 LLM 호출 직렬화 · 모든 호출은 usage_scope로 phase/step 자동 태깅

미션 1. 데이터 파이프라인 · [수행]

비정형 PDF/이미지 → 구조화 JSON → 12가치 매핑

파이프라인 4단계

01 OCR

Claude Vision 기반. "위→아래, 좌→우, 2-컬럼 블록 유지"
읽기 순서를 프롬프트로 강제.

02 익명화 (PII-safe)

이메일·전화번호를 토큰 치환.
분석 후 복원 — 프롬프트 오염 차단.

03 구조화 추출

6필드 JSON: career / education / skills /
achievements / certifications / projects.

04 12가치 매핑

가치별 score(0–100) · evidence(1–2문장) ·
examples(원문 인용 배열) 생성.

프롬프트 엔지니어링 디테일

- 부정 예시 명시 — "열정적입니다", "책임감이 강합니다" 같은 추상적 자기서술은 증거로 인정하지 않음.
- 80점 하한 임계선 — 충분한 구체적 증거(수치·프로젝트·결과)가 반복 관찰될 때만 80↑ (100점까지 열려 있음).
- 거절 기본값 — 증거가 없는 가치는 0–30 부여하고 examples는 빈 배열.
- 원문 인용 강제 — examples는 이력서의 구체 문장을 원문 그대로 1–3개 인용.

미션 1. 데이터 파이프라인 · [결과]

구조화 데이터셋 + 12가지 점수 + 원문 증거가 한 객체로

결과물

DB 스키마

Analysis.structured_data (6필드 JSON)
Analysis.values_scores (12가지별 점수+근거)

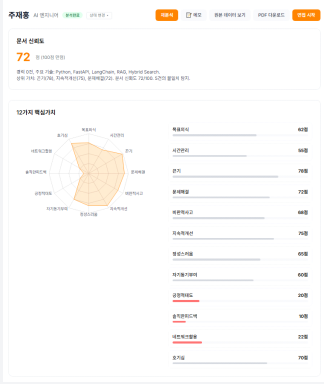
API

GET /api/candidates/{id}/analysis
GET /api/candidates/{id}/debug/raw

프론트 컴포넌트

ValueRadarChart · ValueList · EvidencePanel · RawDataViewer

```
values_scores.지속적개선 = {
  "score": 88,
  "examples": ["ONNX 가속으로 추론 70% 단축", ...]
}
```



리포트 화면 — 12가지 레이더 + 원문 증거

미션 2. 모순 탐지 + 꼬리질문 · [수행]

LangGraph 두 개 — 사전 분석 그래프 + 실시간 면접 그래프

체인 ① 분석 그래프 (사전, analysis_graph.py)

parse_documents	anonymize_pii	extract_structured	score_12_values	**self_verify_evidence**	doc_reliability	**detect_contradictions**	preemptive_questions	compile_restore

모순 탐지 4축: 날짜/기간 · 직책/역할 · 성과 수치 · 기술 스택 · severity high/medium/low

사전 압박 질문: 모순 1건당 1문항+ · 40점 미만 가치 드릴다운 · 개방형 질문 금지

체인 ② 면접 그래프 (실시간, interview_graph.py)

prepare_context	analyze_answer	**generate_followups**	rank_and_filter

question_source 분기

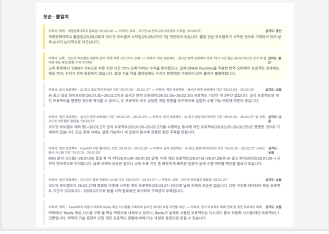
pregenerated / custom / followup
→ 꼬리질문의 꼬리질문도 생략하지 않도록
프롬프트에 출처를 강제 주입.

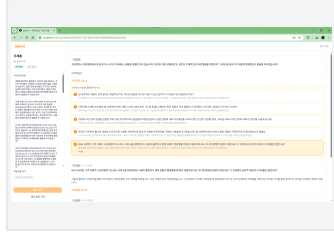
asyncio.Queue 단일 워커

services/followup_worker.py
→ 동시 답변 제출에도 LLM 호출이 직렬화
순서 꼬임·중복 차단.

미션 2. 모순 탐지 + 꼬리질문 · [결과]

시나리오별 입력 → 실제 질문 샘플





질문 샘플 — 모든 질문에 basis (근거) · target_value (연결 가치) 필드 포함

- "이력서에는 99.1% 정확도를 750회 실험으로 달성했다고 되어 있는데, 실험 1회당 평균 학습 시간과 GPU 사양을 알려주실 수 있나요?"
- "ONNX 가속으로 70% 단축이라 적혀 있는데, 단축 측정의 baseline이 무엇이고 어떤 디바이스에서 측정했나요?"
- "답변에 "팀 전체가 같이 했다"가 등장 → 자동 꼬리질문: "본인이 단독 의사결정한 지점은 어디였나요?""

모순 탐지 결과

모순 · 불일치

이력서: 학력 - 계명문화대학교 졸업일 '25.08.08' ↔ 이력서: 교육 - 코드잇 AI 엔지니어 부트캠프 시작일 '25.08.07' **심각도: 중간**

계명문화대학교 졸업일(25.08.08)과 코드잇 부트캠프 시작일(25.08.07)이 1일 역전되어 있습니다. 졸업 전날 부트캠프가 시작된 것으로 기재되어 있어 날짜 순서가 논리적으로 어긋납니다.

이력서: 교육 - 코드잇 부트캠프 졸업작 성과 '추론 지연 시간 70% 단축' ↔ 이력서: 개인 프로젝트 - 실시간 번역 오버레이 성과 '대기 시간 없는 실시간 텍스트 인식 및 번역 성능 확보(정량 수치 미기재)' **심각도: 중간**

교육 항목에서 'ONNX 가속으로 추론 지연 시간 70% 단축'이라는 수치를 명시했으나, 실제 ONNX Runtime을 적용한 번역 오버레이 프로젝트는 성과에는 해당 70% 수치가 전혀 등장하지 않습니다. 동일 기술 적용 결과임에도 수치가 한쪽에만 기재되어 있어 출처가 불명확합니다.

이력서: 공식 프로젝트 - AI 광고 생성 파이프라인 기간 '26.01.30 - 26.02.27' ↔ 이력서: 개인 프로젝트 - 실시간 번역 오버레이 기간 '26.02.05 - 26.02.25' **심각도: 낮음**

AI 광고 생성 파이프라인(26.01.30~26.02.27)과 실시간 번역 오버레이(26.02.05~26.02.25) 프로젝트 기간이 약 3주간 겹칩니다. 공식 프로젝트와 개인 프로젝트를 병행한 것으로 해석할 수 있으나, 두 프로젝트 모두 상당한 개발 분량을 요구하므로 실질적 수행 가능 여부에 의문이 생깁니다.

이력서: 개인 프로젝트 - 실시간 번역 오버레이 기간 '26.02.05 - 26.02.25' ↔ 이력서: 공식 프로젝트 - AI 광고 생성 파이프라인 기간 '26.01.30 - 26.02.27' 및 교육 - 코드잇 부트캠프 졸업일 '26.02.27' **심각도: 낮음**

코드잇 부트캠프 제작 중(~26.02.27) 공식 프로젝트(26.01.30~26.02.27)를 수행하는 동시에 개인 프로젝트(26.02.05~26.02.25)도 병행한 것으로 기재되어 있습니다. 단순 중복 자체는 설명 가능하나 세 일정이 동시에 진행된 점은 주목할 만합니다.

이력서: 개인 프로젝트 - FastAPI 기반 웹서비스 기간 '26.01.13 - 26.01.28' ↔ 이력서: 공식 프로젝트 - AI 광고 생성 파이프라인 기간 '26.01.30 - 26.02.27' / RAG 분석 시스템 기간 '25.12.10 - 26.01.05' **심각도: 낮음**

RAG 분석 시스템(~26.01.05) 종료 후 약 1주(26.01.06~26.01.12) 공백, 이후 개인 프로젝트(26.01.13~26.01.28)와 AI 광고 파이프라인(26.01.30~) 시작이 연속적으로 이어집니다. 날짜 자체의 모순은 없으나 교육 수료 기간 중 빠듯하게 배치된 일정이 실제 수행 여부를 확인할 필요가 있습니다.

이력서: 개인 프로젝트 - 디스코드 홍보 자동화 시스템 기간 '26.03.07 - 26.03.27' ↔ 이력서: 교육 - 코드잇 부트캠프 졸업일 '26.02.27' **심각도: 낮음**

코드잇 부트캠프가 26.02.27에 종료된 이후에 시작된 개인 프로젝트(26.03.07~)이므로 날짜 자체의 모순은 없습니다. 다만 구조화 데이터의 해당 프로젝트 기간이 '2026.03 - 2026.03'으로 동일 시작·종료로만 표기되어 구체성이 부족합니다.

이력서: 학력 - 'FastAPI의 비동기 처리와 Redis 캐싱 시스템을 구축하여 실시간 데이터 요청 지연을 개선' ↔ 이력서: 공식 프로젝트 목록 전체 및 개인 프로젝트 목록 **심각도: 낮음**

약력에서 'Redis 캐싱 시스템 구축'을 핵심 역량으로 내세우고 있으나, Redis가 실제로 사용된 프로젝트는 디스코드 홍보 자동화 시스템(개인 프로젝트) 1건뿐입니다. 약력의 기술 표현이 단일 개인 프로젝트 경험에 비해 다소 과장된 표현으로 읽힐 수 있습니다.

사전 압박 질문

사전 압박 질문

1. **계명문화대학교 졸업일이 2025년 8월 8일이고, 코드잇 부트캠프 시작일이 2025년 8월 7일로 기재되어 있습니다. 졸업 전날 부트캠프가 시작된 것인데, 실제로 입학 처리는 언제 이루어졌고, 졸업식 혹은 학위 수여일이 별도로 존재한다면 정확한 날짜가 언제인지 말씀해 주시겠습니까?**
   계명문화대학교 졸업일(25.08.08)과 코드잇 부트캠프 시작일(25.08.07)의 1일 역전되어 있어 시간은 논리 모순이 발생함.

2. **'추론 지연 시간 70% 단축'이라는 수치가 성과 항목에 기재되어 있는데, 이 수치의 기준 측정값(baseline)은 몇 ms 또는 초였고, 최적화 후 수치는 정확히 얼마였습니까? 또한 이 수치가 번역 오버레이 프로젝트에서 나온 것인지, 다른 프로젝트에서 나온 것인지 구체적으로 어느 프로젝트의 어느 실험에서 측정된 값인지 말씀해 주시겠습니까?**
   성과 항목에 'ONNX 가속으로 추론 지연 시간 70% 단축' 수치가 명시되어 있으나, 실제 ONNX Runtime을 적용한 번역 오버레이 프로젝트 성과에는 해당 수치가 전혀 등장하지 않아 출처가 불명확함.

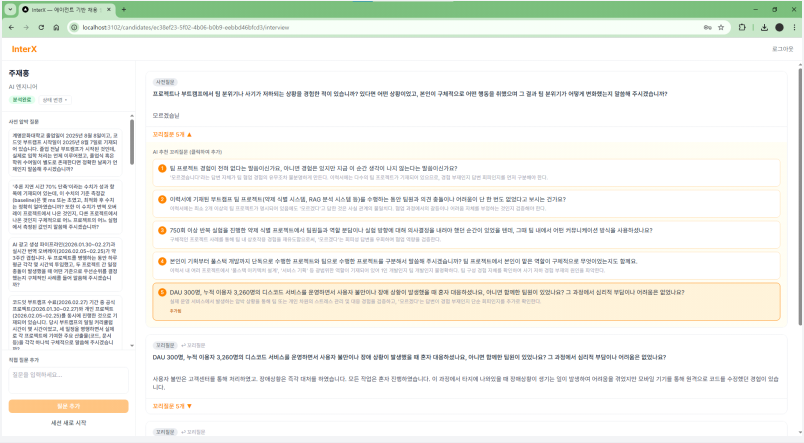
3. **AI 광고 생성 파이프라인(2026.01.30–02.27)과 실시간 번역 오버레이(2026.02.05–02.25)가 약 3주간 겹칩니다. 두 프로젝트를 병행하는 동안 하루 평균 각각 몇 시간씩 투입했고, 두 프로젝트 간 일정 충돌이 발생했을 때 어떤 기준으로 우선순위를 결정했는지 구체적인 사례를 들어 말씀해 주시겠습니까?**
   공식 프로젝트(AI 광고 생성)와 개인 프로젝트(번역 오버레이)의 개발 기간이 3주간 중복되어 실질적 수행 가능 여부에 의문이 있음.

4. **코드잇 부트캠프 수료(2026.02.27) 기간 중 공식 프로젝트(2026.01.30–02.27)와 개인 프로젝트(2026.02.05–02.25)를 동시에 진행한 것으로 기재되어 있습니다. 당시 부트캠프의 일일 커리큘럼 시간이 몇 시간이었고, 세 일정을 병행하면서 실제로 각 프로젝트에 기여한 주요 산출물(코드, 문서 등)을 각각 하나씩 구체적으로 말씀해 주시겠습니까?**
   부트캠프 재학 중 공식 프로젝트와 개인 프로젝트를 동시에 수행한 것으로 기재되어 세 일정이 동시에 진행된 점에서 실제 수행 여부 확인이 필요함.

5. **기술스택에 Tkinter가 명시되어 있으나, 이력서의 어느 프로젝트에서도 Tkinter 사용 기록이 없습니다. Tkinter를 실제로 사용한 프로젝트가 있다면 그 프로젝트명, 개발 시점, 구현한 기능이 무엇인지 구체적으로 말씀해 주시겠습니까?**
   기술스택에 Tkinter가 명시되어 있으나 이력서 내 어떤 프로젝트에서도 Tkinter를 사용한 기록이 없어 실제 경험 여부가 불분명함.

6. **'FastAPI 비동기 처리와 Redis 캐싱 시스템 구축으로 실시간 데이터 요청 지연을 개선했다'고 약력에 기재되어 있는데, 실제로 Redis를 사용한 프로젝트가 디스코드 홍보 자동화 시스템 1건뿐입니다. 해당 프로젝트에서 Redis를 도입하기 전 지연 시간은 몇 ms였고, 도입 후 몇 ms로 개선되었으며, Redis에서 어떤 데이터를 캐싱 키로 설계했는지 말씀해 주시겠습니까?**
   막대에서 Redis 캐싱 시스템 구축을 핵심 역량으로 서술하고 있으나 실제 사용 프로젝트는 1건에 불과하며 표현이 과장일 가능성이 있음.

실시간 면접 화면


InterX — 에이전트 기반 채용
localhost:3102/candidates/ec38ef23-5f02-4b06-b0b9-eebbd46bfcd3/interview
InterX
로그아웃
주재홍
AI 엔지니어
분석완료
상태 변경
사전 설계 질문
맥쿼리정보컨설팅에서 클럽장이 2023년 8월 8일부고, 큰 드인 부리온즈 시작일이 2020년 9월 2일로 기재되어 있는데, 중간 2년간 경력증명서 시작일이 2020년, 실제로 입학 하셨는 건데 이후에 대표, 클럽장 특강 학원 수여일이 별도로 존재한다면 정확한 실제기 인 해당되지 않을때 주시겠습니까?
"추론 지연 시간 70% 단축에 이러는 수치가 성과 항목에 기재되어 있는데, 이 수치의 기준 측정값 (baseline)은 뭐 ms 또는 초였고, 현재를 두 수치는 정확히 얼마였는지?" 면접 이 수치가 병목 상태에 있는 것인지, 구체적으로 어떤 프로젝트에서 나온 것인지 구체적으로 말해 주시겠습니까?
AI 공고 상상 파이프라인(2026.01.30~02.27)과 실시간 번역 오케스트라(2026.02.05~02.25)가 약 3주간 겹칩니다. 두 프로젝트를 병행하는 방안 하루 평균 각각 몇 시간씩 투입했고, 두 프로젝트 간 일정 충돌이 발생했을 때 어떤 기준으로 우선순위를 정했는지 구체적인 사례를 들어 말해 주시겠습니까?
코드 부트캠프 수료(2020.02.27) 직전 출 공식 프로젝트(2020.01.30~02.27)와 개인 프로젝트(2020.02.05~02.21)를 동시에 진행한 것으로 기재되어 있습니다. 당시 부트캠프의 일정 커리큘럼상 시간이 꽉 시간이었고, 제 일정을 병행하면서 실제로 각 프로젝트에 기여한 주요 산출물(코드, 문서, 팀원을 도와 나눠 구체적으로 말해 주시겠습니까?
직접 질문 추가
질문을 입력하세요...
질문 추가
세션 새로 시작
사전질문
프로젝트나 부트캠프에서 팀 분위기나 사기가 저하되는 상황을 경험한 적이 있습니까? 있다면 어떤 상황이었고, 본인이 구체적으로 어떤 행동을 취했으며 그 결과 팀 분위기가 어떻게 변화했는지 말해 주시겠습니까?
모르겠습니다
꼬리질문 5개 ▲
AI 추천 꼬리질문 (클릭하여 추가)
1 팀 프로젝트 경험이 전혀 없다는 말씀이신가요, 아니면 경험은 있지만 지금 이 순간 생각이 나지 않는다는 말씀이신가요?
"모르겠습니다"라는 답변 자체가 팀 협업 경험이 유무조차 불분명하게 만든다고. 이력서에는 "다수의 팀 프로젝트가 기재되어 있습니다, 협업 부재인지 답변 회피인지를 먼저 구분해야 한다.
2 이력서에 기재된 부트캠프 팀 프로젝트(깃허 식별 시스템, RAG 분석 시스템 등)를 수행하는 동안 팀원과 직접 출퇴근이나 어려움이 단 한 번도 없었다고 보시는 건가요?
이력서에는 최소 2개 이상의 팀 프로젝트가 명시되어 있음에도 "모르겠다"고 답한 것은 사실 관계에 불일치다. 협업 과정에서의 갈등이나 어려움 자체를 부정하는 것인지 검증해야 한다.
3 750의 이상 반복 실험을 진행한 역재 식별 프로젝트에서 팀원들과 역할 분담이나 실험 방향에 대해 의사결정을 내리야 했던 순간이 있었을 텐데, 그때 팀 내에서 어떤 커뮤니케이션 방식을 사용하셨나요?
구체적인 프로젝트 사례를 통해 팀 내 갈등 또는 협업을 해결하려 했는지를 묻는다. "모르겠다"는 회피나 갈등을 무뢰하게 말한 역량을 검증한다.
4 본인이 기획부터 출스택 개발까지 단독으로 수행한 프로젝트와 팀으로 수행한 프로젝트를 구분해서 말씀해 주시겠습니까? 팀 프로젝트에서 본인이 맡은 역할이 구체적으로 무엇이었는지도 함께요.
이력서 내 대다 프로젝트에서 '출스택 파이프라인 설계', '시리즈 기획' 등 단일 인원 작업이 기재되어 있어 이 역할인지 팀 역할인지 명확하지 않다. 팀 구성 정보 자체를 확인하여 사기 저하 요인 부재와 원인을 파악한다.
5 DAU 300명, 누적 이용자 3,260명의 디스크드 서비스를 운영하면서 사용자 불만이나 장애 상황이 발생했을 때 혼자 대응하셨나요, 아니면 팀원한 팀원이 있었나요? 그 과정에서 심리적 부담이나 어려움은 없었나요?
실제 운영 서비스에서 발생하는 장애 상황을 통해 팀 유무는 개인 복원력 소진과, 번아웃 등 대응 경험을 검증하는, 이력서에서 가장 중요한 부재의 진실 확인 질문이다.
추가됨
꼬리질문
↳ 꼬리질문
DAU 300명, 누적 이용자 3,260명의 디스크드 서비스를 운영하면서 사용자 불만이나 장애 상황이 발생했을 때 혼자 대응하셨나요, 아니면 팀원한 팀원이 있었나요? 그 과정에서 심리적 부담이나 어려움은 없었나요?
사용자 불만은 고객센터를 통해 처리하였고, 장애상황은 즉각 대처를 하였습니다. 모든 작업은 혼자 진행하였습니다. 이 과정에서 타치게 나타났을 때 장애상황이 생기는 일이 발생하여 어려움을 겪었지만 모바일 기기를 통해 원격으로 코드를 수정했던 경험이 있습니다.
꼬리질문 5개 ▼
꼬리질문
↳ 꼬리질문

미션 3. AX 대시보드 · [수행]

후보자 등록 → 분석 → 리포트 → 면접 → 합격 결정까지 한 화면

아키텍처

백엔드	FastAPI (port 8102) 라우터 분리: auth · candidates · documents · analysis · interview
프론트	Next.js 14 App Router · TypeScript · Tailwind (port 3102)
저장소	SQLite + SQLAlchemy
PDF	reportlab 서버사이드 · 한글 폰트(Malgun Gothic) 자동 등록

안정성

call_claude_json 2회 재시도 · current_step 기반 재개 · API_SLEEP_SECONDS=0.5s로 rate limit 완충

핵심 컴포넌트 맵

- **ValueRadarChart** — 12가치 레이더 시각화
- **ValueList + EvidencePanel** — 가치 클릭 → 원문 증거 슬라이드아웃
- **ContradictionList** — 모순 4축 + 심각도 색상 강조
- **PreemptiveQuestions** — 근거(basis)·연결 가치 함께 노출
- **StatusSelector** — 9단 상태(미분석~최종합격) 즉시 변경
- **ProgressBar** — 단계별 진행 + 예상 잔여 시간
- **RawDataViewer** — OCR 원문 ↔ 구조화 JSON 탭 비교
- **면접관 메모** — 리포트 상단 1클릭 · 후보자 비노출

미션 3. AX 대시보드 · [결과]

동작하는 웹 프로토타입 — 5분 시연

▶ 시연 영상

엔지니어링 고민 ① 할루시네이션 자가검증 루프

"근거가 원문에 존재하는가"를 기계가 스스로 확인하게 만들었습니다

문제 정의

LLM이 examples 필드에 원문에 없는 문장을
만들어 넣는 환각.
→ 면접관이 리포트를 신뢰할 수 없는
치명적 실패 모드.

검증 루프

score_12_values	**self_verify_evidence**
regenerate (필요 시)	doc_reliability

4층 방어선

01 프롬프트 제약

추상적 자기서술 불인정 · 원문 문장 그대로 인용 강제.

02 근거 필드 강제

evidence · examples · basis · reasoning 전부 스키마 필수.

03 자가 점증 검증 루프 (핵심)

substring + token-overlap ≥ 0.6 · 점수 40↑ & 검증 비율 < 50% →
해당 가치만 1회 재호출해 인용 교체. verification 필드로 DB 영속화.

04 사람 검증 UI

RawDataViewer — OCR 원문 ↔ 구조화 JSON 나란히 비교.

엔지니어링 고민 ② 토큰·비용 실측 + Claude Code 활용

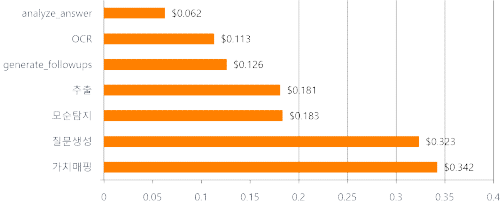
대략치 메모가 아니라 DB에 남는 수치로 비용을 본다

비용 계측 설계

token_usage 테이블 · contextvars 자동 태깅

candidate_id · phase · step · model · input/output tokens · cost_usd
usage_scope(candidate_id, phase, step) 컨텍스트로
LangGraph 노드 · OCR · Vision 호출이 자동 태깅되어 DB 기록.

실측 — step별 누적 비용 (USD, N=3 분석)



실측 요약 (N=3 분석, claude-sonnet-4-6)

$0.38

분석 1회 평균 · 분석 단계 누적 $1.14 / 19 호출

면접(8 호출) $0.19 · 꼬리질문 1회 $0.03–0.05

최신 AI 도구 활용 — Claude Code Cli

claude-opus-4-7 을 프로그래밍 모델로 사용

- LangGraph 노드 분리 설계 검토
- 자가검증 루프 임계값(token-overlap 0.6) 결정 토론
- reportlab 한글 폰트 등록 트러블슈팅
- 본 발표 자료(스크립트·차트 명세) 초안 생성

솔직히 못 한 부분 및 개선하고 싶은 부분

보강해야 할 4가지 — 어떤 순서로 풀 것인지

미흡 항목과 그 이유

① 프롬프트 회귀 테스트 부재

고정 픽스처 × 계약 검증을 pytest 스위트로 만들지 못함. MVP 우선순위에서 밀림.

② Prompt Cache 미적용

시스템 프롬프트·12가치 정의를 Anthropic prompt cache로 고정 못함. 토큰 절감 여지.

③ 다중 평가자 협업·권한 모델 미구현

단일 사용자 시나리오에 집중. 평가자별 권한·교차 평가·합산 합격 흐름 미구현.

④ LLM 평가 자동화(Eval) 부재

Ragas/LangSmith 같은 도구로 모순·꼬리질문 품질을 정량화 못함. 사람 검수에 의존.

개선하고 싶은 부분

개선사항 1

① 해소.
고정 이력서 픽스처 5종 × 계약 검증 pytest 스위트 + CI.

개선사항 2

② 해소.
Prompt cache를 시스템·12가치·OCR 지침에 적용 →
토큰 비용 약 30% 추가 절감.

개선사항 3

④ 첫 걸음.
모순·꼬리질문 회귀용 골든 데이터셋 30건 +
자동 평가 파이프라인 초안.

무엇을 못 했는지 알고 있다는 것 + 어떤 순서로 풀 것인지가 정해져 있다는 것.

한 줄 메시지

돌아가는 AI가 아니라,

분기마다 더 싸지고 더 정확해지는 AI를 만들겠습니다.

- 0.1%의 효율을 측정 가능한 인프라 위에서 쌓아가겠습니다.
- 인터엑스의 12가지 가치를 회귀 테스트·비용 계측·자가 검증 루프로 옮기겠습니다.

감사합니다.

주재홍 · kinduki123@gmail.com · github.com/flyingjoojak